

Cybercrime and Fraud Detection

Final Project Work

Giliano Amadei
256171

Clara Maria Ferracini
262731

Riccardo Paolantoni
262341

Abstract—In this project, our main objectives are to identify a counter-fraud strategy by employing Machine Learning models on our dataset. Specifically, we aim to identify the distinguishing characteristics of fraudulent links and develop a classification system capable of differentiating between fraudulent and non-fraudulent links and also create a classification model that is able to distinguish between different types of compromising links like phishing and defacement. By achieving this goals, we can enhance future fraud detection and prevention measures. The models we employ are Naive Bayes, Logistic Regression, and Decision Tree.

I. INTRODUCTION

A hidden fraudulent URL is an URL to an illegitimate web page added to a trusted website. A trusted website is a site whose administrators perform their best effort to host content that is indeed genuine and not harmful. A compromised website is a site that hosts both fraudulent and legitimate (i.e., not fraudulent) web pages. A hidden URL is an URL of a page that is not reached by crawling the corresponding website within up to the third level of depth. Given this, we identified our counter-fraud goal in finding what are the characteristics that differentiate URLs that are either hidden fraudulent or belonging to compromised web pages, from URLs that are not compromised or fraudulent. In order to do it we started with an exploratory data analysis (EDA) to analyze the dataset. EDA uses a variety of methods and visualizations to analyze the data more deeply and find any significant trends, patterns, or anomalies. We can find significant details, outliers, distributions, and other data properties using EDA that may be pertinent to our fraud detection work. With the use of the correlation matrix analysis and EDA, we hope to better comprehend the relationships between the variables in our dataset and pinpoint the distinctive characteristics of fake linkages. Then we proceeded with a preprocessing phase of our dataset to remove all irrelevant values for each attribute. We have also handled missing and we have defined the positive class that we are specifically interested in predicting. This positive class represents the instances or links that are considered fraudulent. By performing these preprocessing steps, we have prepared the dataset for further analysis and modeling to develop a predictive model for fraud detection. For what concerns the analytical phase, in order to determine the correlation coefficients between our variables, we did a correlation matrix. After doing the exploratory data analysis, the preprocessing and correlation analysis, we used RapidMiner's AutoModel

function to apply three distinct classification models to our data: Naive Bayes, Logistic Regression, and Decision Tree.

The main limitation of our study is the general imbalance of the dataset, and in particular for the variable *isHiddenFraudulent*, since the two classes TRUE and FALSE have a big difference in observations, as we will later see in the EDA part. The modeling and analysis procedure may be complicated by this considerable class imbalance. Machine learning models can perform and be more accurate when given balanced datasets, but they can also be more biased in favor of the dominant class. In our example, the authentic URLs make up the majority class, while the fraudulent URLs make up the minority class. The models may struggle to effectively capture the patterns and traits connected to fraudulence due to the small number of examples in the fraudulent class. This is also why we decided to analyse, together with *isHiddenFraudulent*, the variable *compromissionType* since, in our opinion, the results are more meaningful and interesting. Although we are aware of this constraint, we will proceed cautiously and look at ways to lessen the effects of class imbalance in our analytic and modeling jobs.

II. DATASET DESCRIPTION

Our dataset is a collection of 185180 urls belonging to three different categories: hidden fraudulent URLs, which are URLs to an illegitimate web page added to a trusted web site, URLs of legitimate pages belonging to trusted, yet compromised web sites and URLs of legitimate pages belonging to trusted and uncompromised web sites. And then we can identify two categories of fraudulent web pages: web defacements and phishing. The dataset is a csv file with the following columns:

- **url**, the actual URL
- **compromissionType**, a categorical value among "phishing", "defacement" or the empty string, if, respectively, the URL belongs to a web site compromised by phishing, defacement or not compromised
- **isHiddenFraudulent**, a boolean value which is true if the URL is hidden and fraudulent
- **contentLength**, an integer value corresponding to the Content-Length header value obtained by sending a HTTP HEAD request to URL
- **serverType**, a string corresponding to the X-Server-Type header value, obtained as above

- **poweredBy**, a string corresponding to the X-Powered-By header value, obtained as above
- **contentType**, a string corresponding to the Content-Type header value, obtained as above
- **lastModified**, an date in the format "Thu, 17 Jan 2013 18:49:04 GMT" corresponding to the Last-Modified header value, obtained as above

III. EXPLORATORY DATA ANALYTICS

Prior to commencing the analysis of the models, we conducted an initial exploratory analysis in R to gain a better understanding of the dataset's characteristics and identify any potential challenges. The first step involved examining the types of variables present in the dataset. By utilizing the *glimpse()* function from the "tidyverse" library, we observed that the majority of variables (attributes) were of the character type. The only exceptions were the boolean variable *isHiddenFraudulent* and the integer variable *contentLength*. Next, we focused on analyzing the distributions of the variables. We began by employing the *summary()* function in R, which provided us with an initial overview of the data's shape. From this step, we inferred that both *contentLength* and *isHiddenFraudulent* exhibited somewhat skewed distributions. Specifically, for *contentLength*, there were 143,245 observations with a value of 0, constituting the first and third quartiles. As for *isHiddenFraudulent*, it displayed an uneven distribution, with the majority of observations belonging to the FALSE class. Another noteworthy variable is *compromissionType*, which shared similarities with *isHiddenFraudulent* in terms of distribution. Similar to the latter, *compromissionType* had the highest number of observations in two out of its three classes. The results are shown in the table below:

isHiddenFraudulent distribution:	
FALSE	TRUE
176471	8709

compromissionType distribution:		
defacement	normal	phishing
96514	3713	84953

Upon examining the remaining variables, we observed a substantial amount of intra-class variance. For instance, variables like *lastModified* exhibited numerous single observations, indicating a wide range of values within each class. In the case of *contentType*, *poweredBy*, and *serverType*, the intra-class variance remained high, but many observations shared similar characteristics. For instance, within *serverType*, there were various instances of Apache (e.g., Apache, Apache/1.3, Apache/2.0, etc.). These patterns of intra-class variance will be taken into account during the preprocessing phase to mitigate their impact. Moving on to the analysis of missing values, initially, the dataset contained only 53 missing values. However, we discovered that numerous observations were represented by empty cells. Consequently, we converted all empty cells into "NA" to accurately identify missing values. Surprisingly,

this transformation resulted in a staggering number of 165,821 missing values. Given the substantial amount of missing values (NA), we conducted an analysis to determine which columns had the highest number of missing values. The results of this analysis are summarized in the following table:

NA per column:			
lastModified	77592	contentType	2306
ServerType	2082	poweredBy	83841

The previous table indicates that out of the eight variables analyzed, four of them contain missing values. Among these four variables, *lastModified* and *poweredBy* have the highest number of missing values. These missing values can pose challenges during data analysis and modeling, requiring appropriate handling strategies. Shifting our focus to duplicates within the dataset, we identified a total of 42,378 rows that can be considered duplicates. These rows share multiple variables with the same values, indicating a replication of information. Duplicate rows can introduce biases and distort the analysis, potentially affecting the performance and accuracy of the models built on the data. To summarize, the preliminary exploratory analysis revealed that most variables were of the character type, except for *isHiddenFraudulent* (boolean) and *contentLength* (integer). The distributions of *contentLength* and *isHiddenFraudulent* appeared to be skewed and uneven, respectively. We found a notable intra-class variance in most of the remaining variables. Variables such as *lastModified* displayed a multitude of single observations, indicating a wide distribution within each class. On the other hand, variables like *contentType*, *poweredBy*, and *serverType* demonstrated high intra-class variance but with several instances sharing similar characteristics. Additionally, we discovered a significant number of missing values (165,821) in the dataset and duplicates (42378). It is important to address both missing values and duplicates during the preprocessing phase to ensure the reliability and quality of the dataset before proceeding with further analysis and model development.

IV. PREPROCESSING

After conducting the initial analysis of the dataset in R, we opted to utilize *RapidMiner Studio* for preprocessing and subsequent analysis. Addressing the first major issue, which involved a large number of missing values within the dataset, we devised different approaches for different attributes. For the *serverType* and *contentType* attributes, we decided to remove all observations with missing values. This decision was made because the number of missing values in these attributes was relatively small, and eliminating these observations would not significantly affect the dataset's overall integrity. However, for the *poweredBy* attribute, which contained the majority of missing values, a different strategy was employed. Instead of completely discarding the attribute, we chose to replace the missing values with the class mean value, which in this case was "PHP." This approach allowed us to retain the information associated with this attribute, even though it introduced some

lack variance due to the imputed values. By imputing the missing values with the mean class value, we aimed to preserve the usefulness of the attribute in the subsequent analysis, despite the presence of missing values. Regarding the *lastModified* attribute, we made the decision to remove it from the dataset since this attribute would not be utilized in the modeling or analysis. This process laid the foundation for further modeling and analysis operations, ensuring the dataset's suitability for subsequent tasks. To address the issue of high intra-class variance in the *contentType* and *serverType* attributes, which had a large number of classes initially, we implemented techniques to standardize and streamline these characteristics. In the case of the *serverType* attribute, each server type had different versions, resulting in numerous categories. To simplify and consolidate the representation of server types, we decided to ignore the version details and group together various versions of Microsoft, Apache, nginx, tfe, and PHP under their respective major names. By eliminating redundant variants, we achieved a more cohesive classification of server types. This approach reduced the number of separate classes, making the attribute values easier to interpret and manage for subsequent analysis. Similarly, for the *contentType* attribute, we divided the values into three major categories: text, picture, and application. However, it was crucial to effectively handle wildcards and inconsistencies present in the dataset, as they could lead to confusion and hinder accurate classification by the models. To overcome this challenge, we meticulously processed the *contentType* values, ensuring a clear separation between the text, image, and application categories. This involved extracting essential data and removing any ambiguity caused by wildcards using techniques such as pattern matching, regular expressions, or specific data manipulation methods. By standardizing and streamlining the *serverType* and *contentType* attributes in this manner, we aimed to enhance the interpretability and efficiency of the subsequent models. Simplifying the attribute values and eliminating unnecessary variants improved the models' ability to classify and analyze the data effectively. Following the categorization of the observations in the two attributes, we utilized the pivot operator in RapidMiner to exclude observations with fractional counts less than 0.01. Here's an overview of the steps we took:

- 1) We selected *serverType* as the pivot attribute for the *serverType* attribute and applied the pivot operator.
- 2) The fractional count of each distinct server type was calculated using the fractional count aggregation method.
- 3) This process resulted in a table containing server types and their corresponding fractional counts.

We repeated the same steps for the *poweredBy* attribute, using the pivot operator and *poweredBy* as the pivot attribute. We calculated the fractional count for each distinct value of *poweredBy* using the fractional count aggregation method. Subsequently, we connected these tables to our initial dataset based on a common attribute (such as *serverType* or *poweredBy*) using a join operation. By executing the join, we were

able to exclude observations with fractional counts below the predetermined threshold of 0.01. This approach allowed us to focus on the more common and significant server types and *poweredBy* values, as we retained only the observations with substantial fractional counts. By discarding observations with lower fractional counts, we were able to prioritize the most representative and meaningful data points in the dataset. After this phase of manipulation we tried to create some new variables to be use into the analysis. We investigated the generation of new variables based on the current data in order to get further insights. The length of the URL was one of these variables that we produced. This strategy was used since extracting information straight from the URL with text mining seemed too difficult due to the URL's complicated structure and plenty of characters. As a result, we assumed that the URL's length could be related to how fraudulent it is. We did it by computing the length of each URL and introducing it as a new attribute. Then, to see whether there was any significant link between the URL length and the goal variable (fraudulence), we looked at the correlation between the two. This investigation would provide light on the possibility that a valuable attribute for identifying counterfeit URLs might be the length of the URL. By adding this additional variable, we broadened the scope of our study and looked at other perspectives in order to look for any patterns or traits that may be connected to fake relationships. By introducing a dummy variable, we further tried to distinguish between URLs that used the HTTP and HTTPS protocols. Hypertext Transfer Protocol is the name of the HTTP protocol, whereas HTTPS is the name of the HTTP protocol Secure. We predicted that this differential would show a link with our target variable, suggesting a possible connection between fraudulence and protocol type. To determine if a link was utilizing the HTTPS protocol, we added a binary value (0 or 1) to the fake variable. HTTPS was represented by a number of 0, whereas HTTP was represented by a value of 1. This investigation would shed light on the possible use of the protocol type as a characteristic for identifying fraudulent URLs. But unfortunately, this new variable did not show to be insightful for our analysis, since the number of observations with value HTTPS were very few and not relevant from a statistical point of view. In machine learning and classification tasks, it's crucial to define the dependent variable and the positive class. It aids in defining the target variable we want to forecast as well as the interest class. So we used RapidMiner's "Define Target" operator to set *isHiddenFraudulent* to the dependent variable. By doing this, we made it clear that the variable *isHiddenFraudulent* is the one whose prediction or classification we are trying to make. All other attributes are also assigned to the role *regular*, which implies that they will be considered independent variables in the study. The model will be trained using these characteristics in order to predict the value of the target variable. Additionally, we declared the *isHiddenFraudulent* variable's positive class to be TRUE which represents the URLs that are fraudulent. This is because our primary interest lies in identifying URLs that are actually fraudulent and by doing that our model will

focus on classifying this class correctly. To further prepare our data for the employment of the models, it is important to convert nominal or categorical variables into numerical representations. We first do that with the operators *Text to Nominal* and *Nominal to Real*, and then with the operator that performs a one-hot-encoding. Encoding is a common method for accomplishing this, which transforms textual or category information into numerical values that may be incorporated into regression models. Finally, after reordering the attributes in alphabetical order and storing the dataset for the further creation of the models we computed the correlation matrix of our preprocessed data. In the RapidMiner process, we made the decision to implement two parallel preprocesses. These two processes share the same steps up until the point of performing one-hot encoding. The reason behind this approach is our specific interest in analyzing two different aspects: the variable *isHiddenFraudulent* as the dependent variable and the variable *compromissionType* as the dependent variable. By creating two distinct final datasets, we can explore the relationship between the independent variables and each of the dependent variables separately. This choice allows us to apply different classification models to each dataset, enabling us to gain valuable insights into the factors contributing to both *compromissionType* and *isHiddenFraudulent*. By taking this approach, we aim to provide a more comprehensive analysis of the data, allowing us to understand the underlying factors influencing both fraudulent activity and the different types of compromises that occur. This way, we can gain a more nuanced understanding of the data and potentially develop more accurate and targeted strategies to detect and prevent fraudulent behavior.

V. NAIVE BAYES

The Naive Bayes method uses the Bayes theorem to classify data in a probabilistic manner. The assumption that characteristics are independent from one another under the conditions of the class label is referred to be "naive" in this context. This presumption makes probability calculations easier to understand and enables effective training and prediction. In practice, characteristics are frequently connected, therefore the naïve assumption might not always be accurate. Naive Bayes can nevertheless perform well in a variety of real-world classification problems despite this oversimplifying assumption. This model is appropriate for our dataset because it is clear from inspecting the correlations matrix that our variables exhibit low correlations. As a result, the model's performance isn't significantly impacted by the assumption of feature independence.

VI. LOGISTIC REGRESSION

Logistic Regression is a statistical model frequently used for categorization and predictive analytics. When the dependent variable is binary or categorical, it is very helpful. By applying a logistic function to the independent variables in a logistic regression, the model calculates the likelihood that an event will occur. Since this probability is confined between 0 and 1,

binary outcomes may be predicted using it. A binary value, such as 0 for the lack of an event and 1, for its occurrence, is frequently used to denote the dependent variable. The linear combination of the independent variables is transformed using the logit formula by the logistic function, sometimes referred to as the sigmoid function. The probability of an event happening are transformed into a linear connection with the predictors using the logit function. The log-odds of an event occurring are affected by each independent variable, and the logistic regression model calculates their coefficients and quantifies their effects. These coefficients can be analyzed to determine the strength and direction of each predictor's influence on the likelihood of the event. The logistic regression technique increases the likelihood of the observed outcomes given the variables during the model training phase. Finding the ideal coefficient values that maximize the likelihood function and provide the greatest fit for the data is the goal of this approach. Once trained, the logistic regression model can be used to predict the probability of the event occurring for new instances based on their independent variable values. A threshold can be applied to convert the probabilities into binary predictions, typically with a cutoff of 0.5. Finding the coefficients' ideal values to maximize the likelihood function and provide the greatest fit for the data is the process at hand.

VII. DECISION TREE

The decision tree is a non-parametric supervised learning algorithm used for classification and regression applications. It is organized hierarchically and has a root node, branches, internal nodes, and leaf nodes. A decision tree has a root node at the beginning that has no incoming branches. The internal nodes sometimes referred to as decision nodes, are fed by the root node's outgoing branches. Both node types undertake assessments based on the available attributes to create homogeneous subsets, which are represented by leaf nodes or terminal nodes. All of the outcomes within the dataset are represented by the leaf nodes. Different groups within an organization can better comprehend why a choice was taken thanks to this form of flowchart layout, which also produces a simple-to-grasp depiction of decision-making. By using a greedy search to find the ideal split points inside a tree, decision tree learning uses a divide-and-conquer technique. When most or all of the records have been categorized under distinct class labels, this splitting procedure is then repeated in a top-down, recursive fashion. The intricacy of the decision tree has a significant role in determining whether or not all data points are categorized as homogeneous sets. Pure leaf nodes, or data points belonging to a single class, are easier to get in smaller trees. It gets harder to preserve this purity as a tree gets bigger, which typically leads to too little data falling under a particular subtree. This is known as data fragmentation, and it frequently results in overfitting. Because of this, decision trees favor tiny trees, which is in line with Occam's Razor's concept of parsimony, which states that "entities should not be multiplied beyond necessity". In other words, decision trees should only increase complexity if absolutely essential. After

all, the simplest explanation is frequently the best. Pruning is typically used to minimize complexity and prevent overfitting; it involves cutting off branches that divide on unimportant traits. Cross-validation may then be used to assess how well the model fits the data.

VIII. CLASSIFICATION ANALYSIS

Once we completed the preprocessing phase and obtained the two distinct datasets for analysis, we decided to utilize the powerful Auto Model feature within RapidMiner. It is an extension to RapidMiner Studio that accelerates the process of model building and validation. One of its key advantages is that it creates processes that can be exported and modified in case of necessity. Auto Model focuses on addressing three major problem classes: prediction, clustering, and outliers. Auto Model assists in data evaluation, suggests relevant models for solving the problem, and facilitates result comparison once the calculations are complete. To begin utilizing RapidMiner's Auto Model, the first step is to select the dataset you want to work with. In our case, we would choose the two datasets that were generated from the preprocessing stage. Next, we need to specify the problem for which we seek an answer. In this scenario, we have a binary classification problem related to *"isHiddenFraudulent"* and a multiclass classification problem related to *"compromissionType."* After specifying the problem, the third step involves selecting the variables to work with. In our case, we removed all variables indicated by the program with a red dot on the side, suggesting their exclusion from the analysis. Finally, the last step is to choose the models we want to apply. In our case, we specifically opted for the three classification algorithms mentioned earlier: Naive Bayes, Logistic Regression and Decision Tree. Once we begin using RapidMiner's automodel feature, we delved into how RapidMiner calculates these models. The automodel uses the same process for each different model, It begins by loading and processing the desired data. In this case, it is the table of the results for the pre-processing for both the IsHiddenFraudulent variable and the compromissionType variable. Then it calculates the average for all numerical columns, the mode for all categorical columns and the minimums and maximums for date columns, if there are any. It also joins these results together, by first joining the average and modes, then joining that table with the minimum and maximum for dates. It then drops the key that was used to join these results and the final result of this step is a single row version of the data.

The next step in the is pre-processing. This step is identical to the pre-processing that we carried out, so we feel that we do not need to expand on it. Following this the automodel filters the rows that have a missing label, which in one case is the IsHiddenFraudulent variable and the other is the compromissionType variable. Following this a sample of the data is saved. Once this data is sampled, the sample is partitioned where 60

The next step was handling text columns, which is an optional step, where the model uses text vectorization to manipulate text columns. After handling text columns, the

model carries out further feature engineering, if necessary. When doing this further feature engineering, a sample of the original training data and is preoptimized. In the preoptimization, the sample is cross-validated and built. When training the desired model here, the sample data is split into a 90% training set split and a 10% validation set split and once it is trained, its confidences are calibrated. Once the model is prepared, its performance is calculated using its classification error. Once this preoptimization is done, the same procedure is repeated on the first sample of the training set. Once the desired model is trained, its confidences are calculated using a Platt scaling approach and a cost sensitive scorer is used. Once the model is applied on the sample, the optimization log, tradeoffs and feature sets are remembered for later use. Then the feature set is applied to the original training set and this new table is remembered. When training the desired model on the original data, the automodel uses the performance logs from this first training attempts to retrain a second model with greater efficiency. Finally, confidence scaling is applied to this second model to optimize the accuracy for a given cost matrix. The next step is to transform the validation and training data using the same processes used in the pre-processing. Next, the variables that play the biggest role in the desired prediction model are identified and it remembers the explained predictions and model-specific weights. The next step encompasses a few processes. Firstly, the automodel attempts to make a lift chart, however, it must be noted that it can only be made for binary classification problems. As a matter of fact, the models trained on the compromissiontype dataset do not return this chart. Using the generate batch node, the dataset can be split into multiple hold-out sets, and the desired model is applied to each hold-out set. The automodel also attempts to add binomial performance measurements to each hold-out set and calculate the average performance of all sets. Finally, a model simulator is created. The final step of the automodel begins by appending the validation and training sets together and then generating statistics on the table. Finally it carries out the desired model in the same method as previously mentioned, without using smaller samples of the entire dataset and the results are returned, as seen in the appendix.

IX. MODEL RESULTS

In this section, we will analyze the results of the models created by RapidMiner for our analysis. To evaluate the performance of these models, we will use the accuracy metric, which measures the fraction of correct predictions made by the models. Formally, accuracy is defined as the number of correct predictions divided by the total number of predictions. First, let's examine the results of the classification on the dataset where *"isHiddenFraudulent"* is used as the dependent variable. The image in the appendix shows that the models achieve an accuracy of almost 100%. While this high accuracy may initially seem promising, in reality it has a very easy explanation. During the EDA phase, we discovered that the variable *"isHiddenFraudulent"* exhibits a significant difference in distribution between the two classes: True and False. Specif-

ically, the False class constitutes 95% of the total observations. This class imbalance poses a challenge for the models, as the limited representation of the True class hampers their ability to differentiate between True and False observations effectively. For example, in the case of Naive Bayes, the model predicted the majority class (False) for all the cases, resulting in a high accuracy close to 100% even if it completely ignore the True observations. It is important to recognize that achieving such high accuracy does not provide us with a practical solution. Merely predicting the same value (False) for every observation does not contribute to formulating an optimal strategy for predicting hidden fraudulent URLs. We need a model that can identify the True class accurately. Among the models evaluated, the decision tree model shows the best performance in predicting the desired result. This slight improvement in accuracy can be attributed to the nature of decision trees, as they are non-linear models capable of making significant splits in the data points. Unlike linear models, decision trees can capture complex patterns and relationships. Additionally, examining the small decision tree provided as output, we observe that the most influential split in the tree relates to the length of the URL. URLs shorter than a certain threshold are more likely to be classified as True (fraudulent). This suggests that URL length plays a crucial role in distinguishing between genuine and fraudulent URLs. Now let's shift our focus to the models with "*compromissionType*" as the dependent variable. In this case, we encounter a multi-class classification problem, as "*compromissionType*" can take three possible values: defacement, normal, and phishing. However, similar to the previous scenario with "*isHiddenFraudulent*," we face an issue due to imbalanced class distribution, resulting in models struggling to accurately predict the "normal" class. The limited number of observations associated with the "normal" value hinders the models' ability to discern this class effectively. Nevertheless, despite the challenges posed by imbalanced values, the models exhibit the ability to differentiate between the "defacement" class (as evident in models like "range1") and the "phishing" class (represented by "range3"). This indicates that the models can successfully distinguish between these two compromise types, even though they struggle with the "normal" class. Similar to the previous analysis, the model with the highest accuracy is the binary tree model used for "*isHiddenFraudulent*." However, there are some notable differences in the variables that contribute to the decision-making process. In addition to "*urlLength*," which was identified as a significant variable in the previous models, the current models also consider variables such as "*poweredBy*" and "*serverType*" as meaningful factors. These variables aid in distinguishing between different types of compromise. The results obtained from this analysis prove to be more informative from an analytical standpoint, as they provide insights into the possibility of implementing a classification model that can discern between various compromise types. While accurately predicting the "normal" class remains a challenge due to its limited representation, the models demonstrate promising capabilities in differentiating between "defacement" and "phishing" compromises.

X. FURTHER WORK

In our assessment, a recommended starting point for further work on this analysis would involve collecting a more diverse and comprehensive dataset. The limited success of the classification models can be attributed, at least in part, to the dataset's shortcomings. By gathering a wider range of data samples that cover various scenarios and encompass a greater representation of the classes, we can enhance the models' performance and robustness. Taking the analysis to the next level could involve exploring more advanced classification models, such as ensemble methods like Random Forest. These models have proven to be highly effective in improving classification accuracy by combining the predictions of multiple individual models. By leveraging the power of ensemble techniques, we can potentially achieve better results compared to simpler linear models like Logistic Regression and Naive Bayes. Furthermore, the application of deep learning techniques could unlock even greater potential for improved performance. Deep learning models, with their ability to learn complex patterns and relationships within the data, often outperform traditional linear models. However, it is important to note that deep learning models and ensemble methods typically require substantial computational resources and longer training times. Adequate hardware infrastructure and computational capabilities must be considered when incorporating these kinds of technologies into the analysis workflow. In addition to expanding the dataset and exploring advanced models, another avenue worth considering is the utilization of text mining techniques on the different URLs. By extracting data and characteristics from the URLs, we can potentially uncover valuable insights and features that contribute to the classification task. Text mining can aid in capturing relevant information such as keywords, domain characteristics, and structural attributes, which can further enhance the accuracy and predictive power of the models.

Attributes	compr...	compr...	conten...	conten...	conten...	powere...	powere...	powere...	serverT...	serverT...	serverT...	serverT...	urlLen...
compromissionType = defacement	1	-0.975	-0.144	0.080	-0.080	-0.363	0.374	-0.199	0.418	-0.144	-0.338	-0.138	0.235
compromissionType = phishing	-0.975	1	0.117	-0.078	0.078	0.362	-0.377	0.204	-0.411	0.148	0.333	0.141	-0.227
contentLength	-0.144	0.117	1	0.394	-0.394	0.217	-0.152	-0.044	-0.243	-0.032	0.340	-0.062	-0.137
contentType = image	0.080	-0.078	0.394	1	-0.993	-0.029	0.029	-0.016	0.041	-0.012	-0.038	-0.011	-0.057
contentType = text	-0.080	0.078	-0.394	-0.993	1	0.029	-0.029	0.016	-0.042	0.012	0.038	0.011	0.057
poweredBy = ASP.NET	-0.363	0.362	0.217	-0.029	0.029	1	-0.793	-0.070	-0.565	-0.051	0.760	-0.108	-0.196
poweredBy = PHP	0.374	-0.377	-0.152	0.029	-0.029	-0.793	1	-0.387	0.565	0.064	-0.571	-0.174	0.244
poweredBy = Phusion Passenger (...)	-0.199	0.204	-0.044	-0.016	0.016	-0.070	-0.387	1	-0.275	-0.025	-0.092	0.623	-0.161
serverType = Apache	0.418	-0.411	-0.243	0.041	-0.042	-0.565	0.565	-0.275	1	-0.200	-0.741	-0.442	0.167
serverType = GSE	-0.144	0.148	-0.032	-0.012	0.012	-0.051	0.064	-0.025	-0.200	1	-0.067	-0.040	0.380
serverType = Microsoft	-0.338	0.333	0.340	-0.038	0.038	0.760	-0.571	-0.092	-0.741	-0.067	1	-0.148	-0.228
serverType = nginx	-0.138	0.141	-0.062	-0.011	0.011	-0.108	-0.174	0.623	-0.442	-0.040	-0.148	1	-0.128
urlLenght	0.235	-0.227	-0.137	-0.057	0.057	-0.196	0.244	-0.161	0.167	0.380	-0.228	-0.128	1

Figure 1- Confusion Matrix for compromissionType

Overview

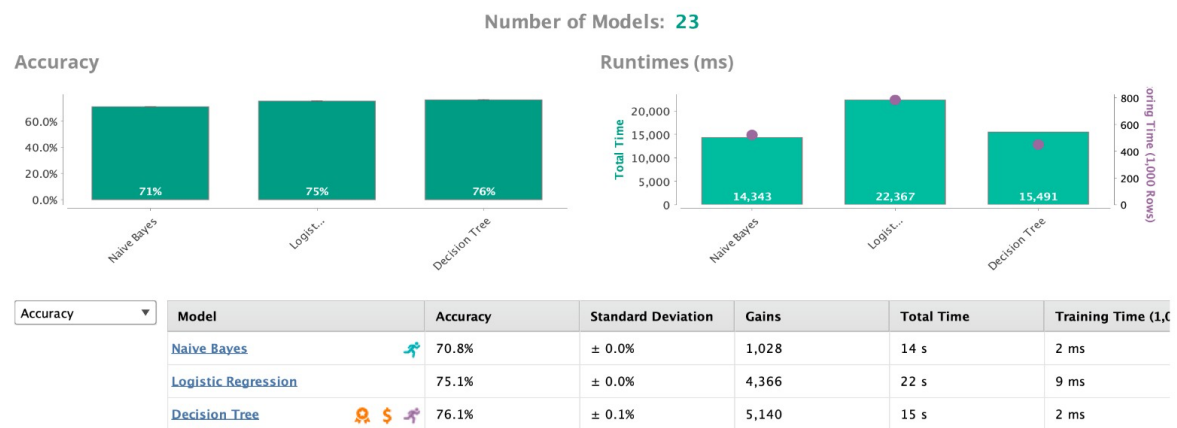


Figure 2 - Models' results for compromissionType

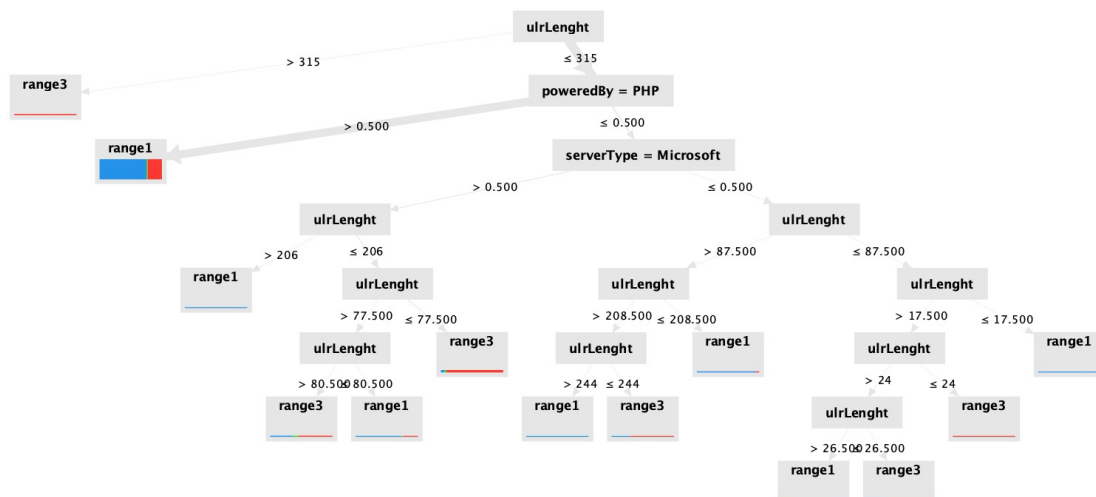


Figure 3 - Decision Tree for compromissionType

Confusion Matrix

	true FALSE	true TRUE	class precision
pred. FALSE	48400	2009	96.01%
pred. TRUE	2	402	99.50%
class recall	100.00%	16.67%	

Figure 4 – Decision tree's confusion Matrix for isHiddenFraudulent

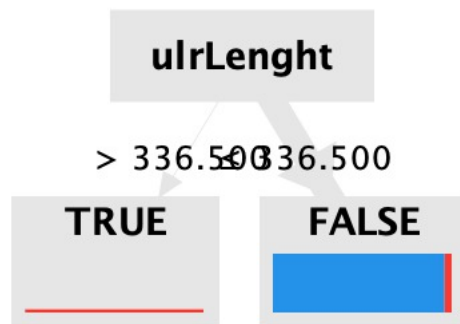


Figure 5 - Decision Tree graoh for isHiddenFraudulent

Attributes	conten...	conten...	conten...	isHidd...	powere...	powere...	serverT...	serverT...	serverT...	serverT...	ulrLen...
contentLength	1	0.457	-0.458	-0.193	0.186	-0.148	-0.184	-0.008	0.249	-0.029	-0.123
contentType = image	0.457	1	-0.993	-0.383	-0.024	0.018	0.030	0.004	-0.032	-0.003	-0.076
contentType = text	-0.458	-0.993	1	0.382	0.024	-0.018	-0.030	-0.004	0.032	0.004	0.075
isHiddenFraudulent = FALSE	-0.193	-0.383	0.382	1	-0.062	0.039	0.144	-0.001	-0.044	0.009	-0.201
poweredBy = ASP.NET	0.186	-0.024	0.024	-0.062	1	-0.830	-0.567	-0.036	0.747	-0.062	-0.165
poweredBy = PHP	-0.148	0.018	-0.018	0.039	-0.830	1	0.440	0.023	-0.602	0.078	0.157
serverType = Apache	-0.184	0.030	-0.030	0.144	-0.567	0.440	1	-0.231	-0.753	-0.458	0.034
serverType = LiteSpeed	-0.008	0.004	-0.004	-0.001	-0.036	0.023	-0.231	1	-0.048	-0.029	-0.025
serverType = Microsoft	0.249	-0.032	0.032	-0.044	0.747	-0.602	-0.753	-0.048	1	-0.094	-0.184
serverType = nginx	-0.029	-0.003	0.004	0.009	-0.062	0.078	-0.458	-0.029	-0.094	1	-0.045
ulrLenght	-0.123	-0.076	0.075	-0.201	-0.165	0.157	0.034	-0.025	-0.184	-0.045	1

Figure 6 - Correlation Matrix for isHiddenFraudulent

Overview

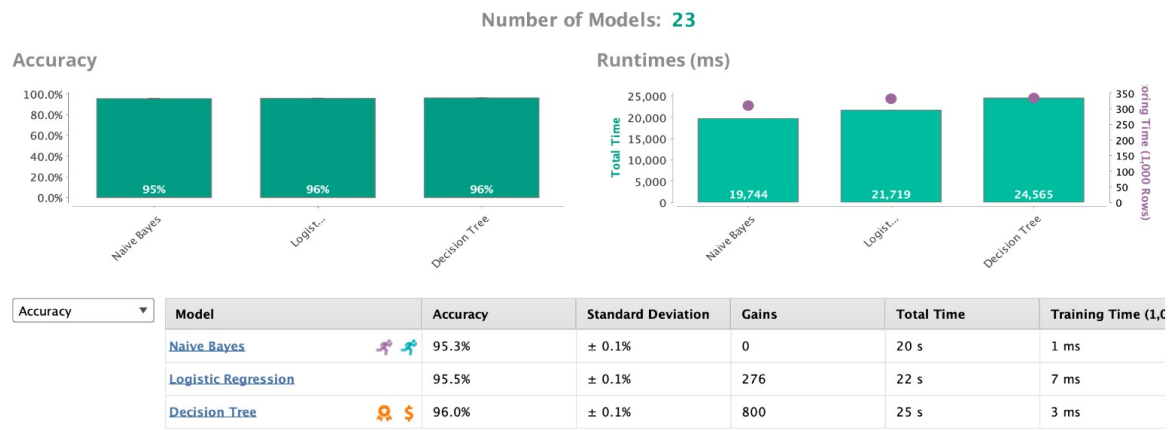


Figure 7 - Models' results for isHiddenFraudulent

Confusion Matrix

	true FALSE	true TRUE	class precision
pred. FALSE	47995	1858	96.27%
pred. TRUE	411	549	57.19%
class recall	99.15%	22.81%	

Figure 8 - Logistic regression's confusion matrix for isHiddenFraudulent

Confusion Matrix

	true FALSE	true TRUE	class precision
pred. FALSE	48408	2406	95.27%
pred. TRUE	0	0	0.00%
class recall	100.00%	0.00%	

Figure 9 - Naive Bayes' confusion matrix for isHiddenFraudulent

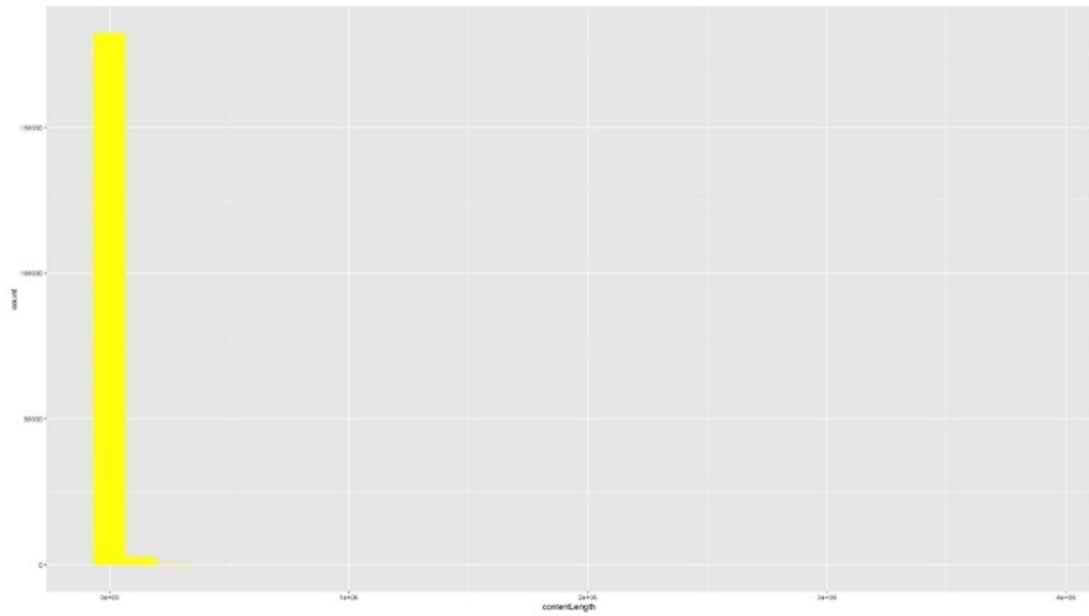


Figure 12 - Distribution of the variable contentLength

ROC Comparison

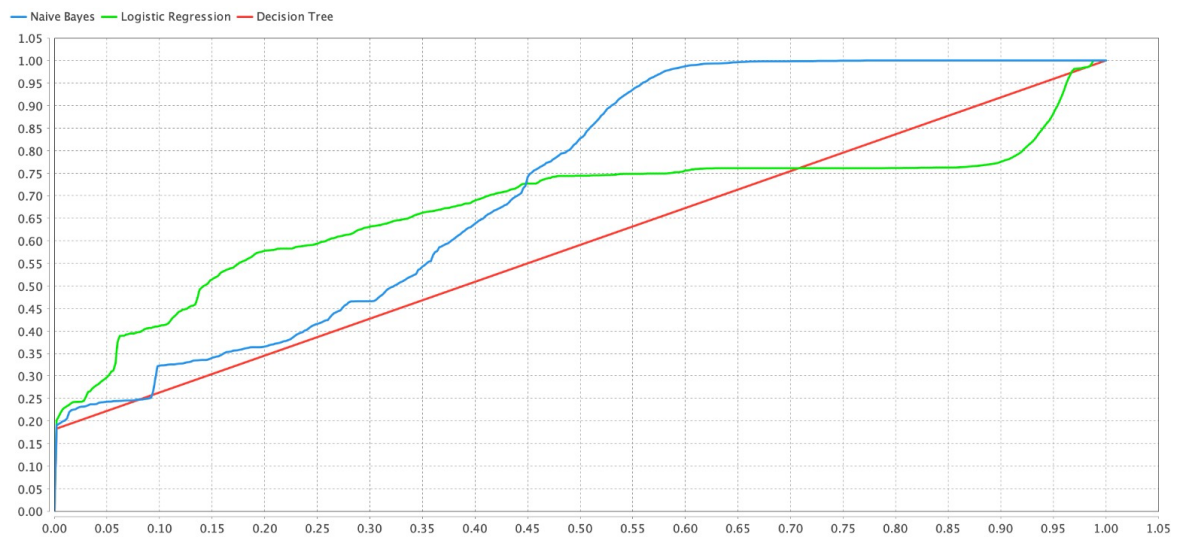


Figure 13 - ROC curve for the models predicting isHiddenFraudulent

Confusion Matrix

	true range1	true range2	true range3	class precision
pred. range1	23694	454	7429	75.04%
pred. range2	0	0	0	0.00%
pred. range3	3271	181	3785	52.30%
class recall	87.87%	0.00%	33.75%	

Figure 14 - Naive Bayes' confusion matrix for compromissionType

Confusion Matrix

	true range1	true range2	true range3	class precision
pred. range1	26547	536	8613	74.37%
pred. range2	0	0	0	0.00%
pred. range3	418	99	2601	83.42%
class recall	98.45%	0.00%	23.19%	

Figure 15 - Logistic Regression's confusion matrix for compromissionType

Confusion Matrix

	true range1	true range2	true range3	class precision
pred. range1	26654	549	8333	75.01%
pred. range2	0	0	0	0.00%
pred. range3	311	86	2881	87.89%
class recall	98.85%	0.00%	25.69%	

Figure 16 - Decision Tree's confusion Matrix for compromissionType